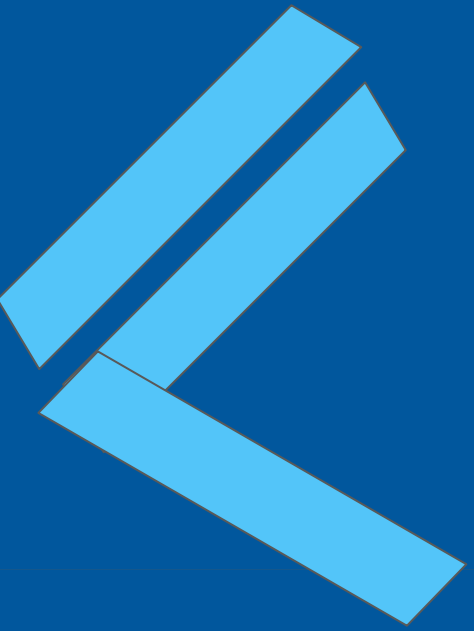


Desenvolvimento Mobile 1
Aulas 07 e 08

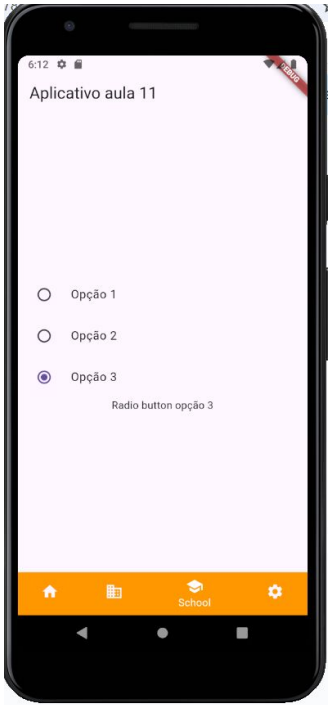
Prof. Me Daniel Vieira

SENAI

Agenda

1 - Navigatorbar, Checkbox, RadioButton, MessageBubble
2 - Layout Chatbot
3 - Projeto integrador com navigator bar e interface do chatbot
4 - Exercício

Flutter Widgets



Flutter Widgets

```
import 'package:flutter/material.dart';

  void main() {
    runApp(NavBotton());
 }
```

Flutter Widgets

```
/*
  Classe NavBotton:
     A classe NavBotton é um StatelessWidget que define a tela princip
Navigator_screen. O MaterialApp contém essa tela como home.
  */
  class NavBotton extends StatelessWidget {
    const NavBotton({super.key});

    @override
    Widget build(BuildContext context) {
      return  MaterialApp(
        home: Navigator_screen(),
      );
    }
  }
```

Flutter Widgets

```
/*
 Classe Navigator_screen:
 Navigator_screen é um StatefulWidget, já que o estado
da tela muda de acordo com o item selecionado no BottomNavigationBar.
 A variável selectIndex controla qual widget será exibido
 na tela principal com base na escolha do usuário no menu de
navegação inferior.
  */
  class Navigator_screen extends StatefulWidget {
   const Navigator_screen({super.key});

   @override
   State<Navigator_screen> createState() => _Navigator_screenState();

 }
```

Flutter Widgets

```
class _Navigator_screenState extends State<Navigator_screen> {
   int selectIndex =0; // variavel pela escolha do widget bar
    // Constante para definir caracteristicas do texto
    static const TextStyle optionStyle =  TextStyle(fontSize: 30,
fontWeight: FontWeight.bold);

  //A lista _widgetOptions contém os diferentes widgets (telas) que
 serão exibidos //com base na seleção do BottomNavigationBar.
       // Cria lista de widgets
    static const List<Widget> _widgetOptions = <Widget>[
     /*Text(
        'Index 0: School',
        style: optionStyle,
      ), */
     // Tela1() ,
     TelaHome(),
```

Flutter Widgets

```
/*Text(
        'Index 1: Business',
        style: optionStyle,
      ),*/
    CheckboxExample(),
     RadioButton(),
      /*Text(
        'Index 2: School',
        style: optionStyle,
      ),
      */
```

Flutter Widgets

```
Text(
        'Index 3: Settings',
        style: optionStyle,
      ),


    ];
```

Flutter Widgets

```
 // O método onItemTapped() altera o valor de selectIndex para refletir o
//selecionado.

    void onItemTapped(int index) {
      setState(() {
       selectIndex = index;


     });
    }
```

Flutter Widgets

```
@override
    Widget build(BuildContext context) {
     return Scaffold(
        appBar: AppBar(
          title: Text("Aplicativo aula 11"),
        ),
        body:  Center(
          child: _widgetOptions.elementAt(selectIndex),
        ),
```

Flutter Widgets

```
/*BottomNavigationBar:
  Contém quatro itens: "Home", "Business", "School" e "Settings", com íco
definidas. O item selecionado é destacado,
  e o conteúdo da tela muda de acordo com a escolha do usuário.
  */
      bottomNavigationBar: BottomNavigationBar(
        items: const <BottomNavigationBarItem>[
          BottomNavigationBarItem(
            icon: Icon(Icons.home),
           label: 'Home',
          backgroundColor: Colors.red,
          ),
```

Flutter Widgets

```
BottomNavigationBarItem(
            icon: Icon(Icons.business),
        label: 'Business',
            backgroundColor: Colors.blue,
          ),
BottomNavigationBarItem(
            icon: Icon(Icons.school),
            label: 'School',
            backgroundColor: Colors.orange,
          ),
```

Flutter Widgets

```
BottomNavigationBarItem(
                icon: Icon(Icons.settings),
                label: 'Settings',
               backgroundColor: Colors.purple,
              ),
            ],
            currentIndex: selectIndex,
            selectedItemColor: Colors.white,
            onTap: onItemTapped,
          ),
              );
      }
  }
```

Flutter Widgets

```
/*
137. Telas Individuais:
138. Tela Home (TelaHome): Uma tela simples com um texto
centralizado, indicando "Mobile 2".
139.
140. */
```

Flutter Widgets - Bottom Navigator

```
class TelaHome extends StatelessWidget {
   const TelaHome({super.key});
 static const TextStyle styletext =  TextStyle(fontSize: 30,
fontWeight: FontWeight.bold);
   @override
   Widget build(BuildContext context) {
     return Scaffold(
                body: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
             Center(child: Text("Mobile 1",style: styletext)),
           ],
        ),
      );
   } }
```

Flutter Widgets - Checkbox

```
/*
160. Checkbox (CheckboxExample): Exibe um Checkbox
que altera o estado de um valor booleano (ischeck)
que é exibido na tela.
161. Radio Button (RadioButton): Exibe três opções
de RadioListTile, e o valor selecionado é armazenado na variável
selectOption.
162.
163. */
```

Flutter Widgets - Checkbox

```
class CheckboxExample extends StatefulWidget {
  const CheckboxExample({super.key});

  @override
  State<CheckboxExample> createState() => CheckBoxState();
}
```

Flutter Widgets - Checkbox

```
class CheckBoxState extends State<CheckboxExample> {
    // variavel para armazenar o valor do check box

    bool ischeck = false;
   @override
   Widget build(BuildContext context) {
     return Scaffold(
       body: Column(
         children: [
           Text("Check Box 1"),
```

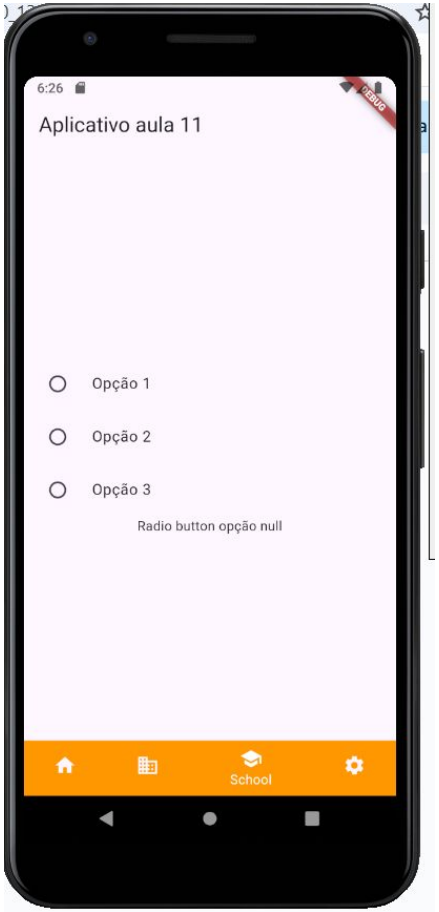
Flutter Widgets - Checkbox

```
Checkbox(
            // Value é o parametro do check box
             // ischeck variavel booleana que recebe o parametro value do check
box
           value:  ischeck,
           onChanged:(bool? value){
             setState(() {
               ischeck = value!;
             });
           }
                                  ),
             Text("Status do checkbox $ischeck"),
          ],
        ),
      );
    }
  }
```

Flutter Widgets - Radio List

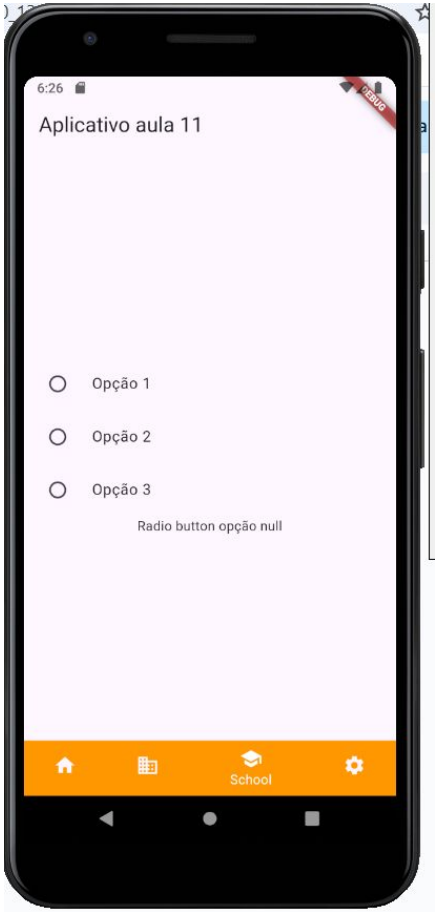
```
class RadioButton extends StatefulWidget {
  const RadioButton({super.key});

  @override
  State<RadioButton> createState() => _RadioButtonState();
}
```



Flutter Widgets - Radio List

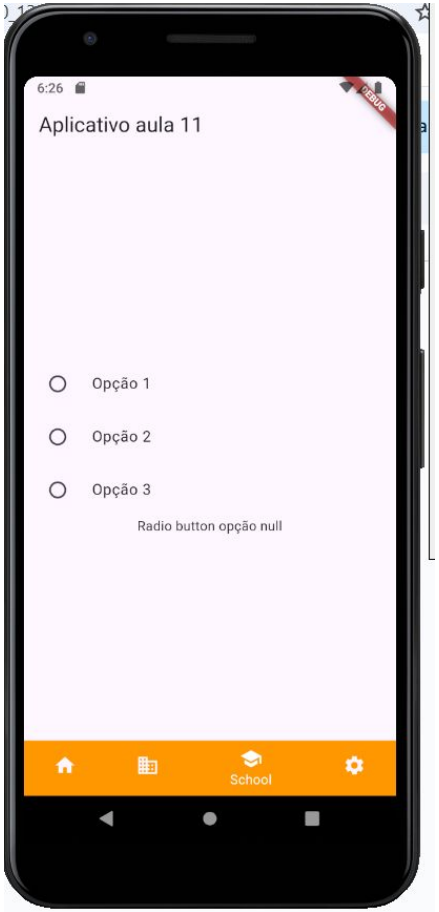
```
class _RadioButtonState extends State<RadioButton> {
  int ? selectOption;
  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.center,
     children: [
        RadioListTile<int>(
          title: Text("Opção 1"),
          value: 1,
          groupValue: selectOption,
          onChanged: (value){
            setState(() {
              selectOption = value;
            });
          },),
```



Flutter Widgets - Radio List

```
RadioListTile<int>(
            title: Text("Opção 2"),
            value: 2,
            groupValue: selectOption,
            onChanged: (value){
              setState(() {
               selectOption = value;

              });
            },  ),
```

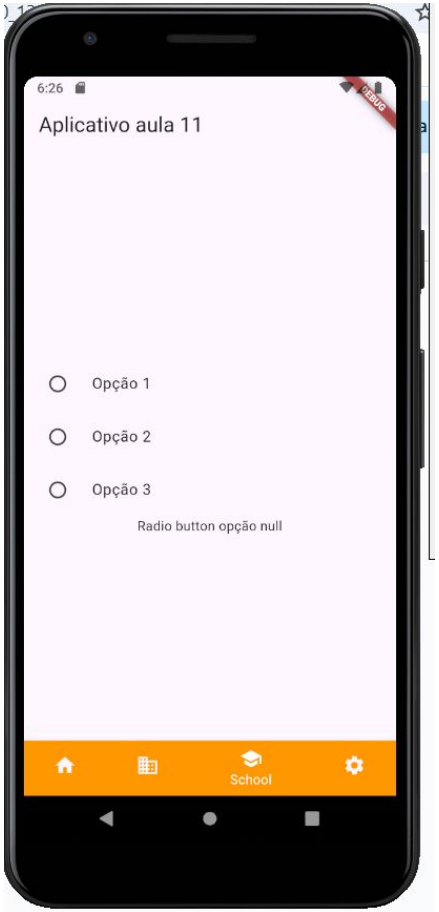


Flutter Widgets - Radio List

```
RadioListTile(
          title: Text("Opção 3"),
          value: 3,
          groupValue: selectOption,
          onChanged: (value){
            setState(() {
              selectOption = value;

            });
          },
          ),
        Text("Radio button opção $selectOption"),

      ],
    );
  }
}
```

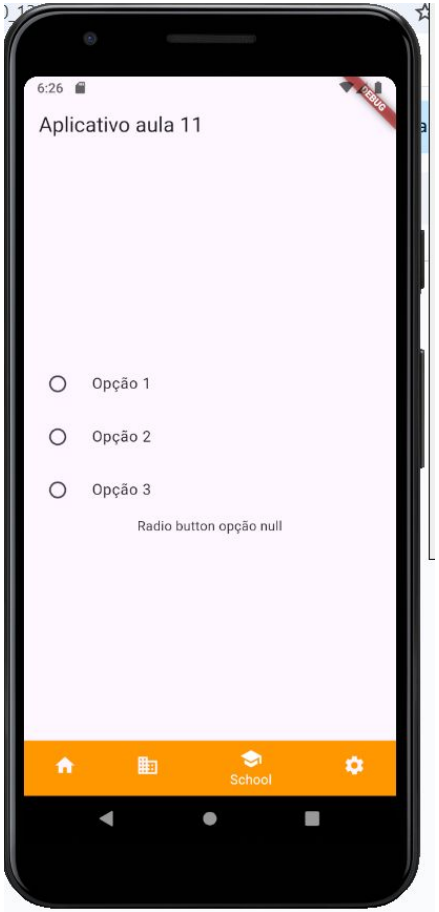


Flutter Widgets - Radio List

```
RadioListTile(
          title: Text("Opção 3"),
          value: 3,
          groupValue: selectOption,
          onChanged: (value){
            setState(() {
              selectOption = value;

            });
          },
          ),
        Text("Radio button opção $selectOption"),

      ],
    );
  }
}
```



Flutter Widgets - Tela ChatBot

```
class ChatScreen extends StatefulWidget {
  const ChatScreen({super.key});

  @override
  State<ChatScreen> createState() => _ChatScreenState();
}
```

Flutter Widgets - Tela ChatBot

```
class _ChatScreenState extends State<ChatScreen> {
  final TextEditingController _controller = TextEditingController();

  final List<Map<String, dynamic>> _messages = [
    {
      'text': "Tipos de solo agricola",
      'isMe': true,
      'time': "5:20 PM"
    },
    {'text': "Calcario, argiloso", 'isMe': true, 'time': "5:20 PM"},
    {'text': "Arenoso", 'isMe': false, 'time': "5:18 PM"},
    {
```

Flutter Widgets - Tela ChatBot

```
  'text': "Qual a melhor semente para plantar em maio",
      'isMe': false,
      'time': "5:18 PM"
    },
    {'text': "Agro IOT", 'isMe': true, 'time': "5:20 PM"},
    {'text': "Projeto integrador", 'isMe': false, 'time': "5:22 PM"},
    {
      'text': "Chatbot",
      'isMe': false,
      'time': "5:30 PM"
    },
  ];
```

Flutter Widgets - Tela ChatBot

```
void _sendMessage() {
    if (_controller.text.trim().isEmpty) return;

    setState(() {
      _messages.add({
        'text': _controller.text.trim(),
        'isMe': true,
        'time': TimeOfDay.now().format(context),
      });
      _controller.clear();
    });
  }
```

Flutter Widgets - Tela ChatBot

```
void _limparMessages(){
  setState(() {
    _messages.clear();
  });
}
```

Flutter Widgets - Tela ChatBot

```
@override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const ListTile(
          contentPadding: EdgeInsets.zero,
          leading: CircleAvatar(
            backgroundColor: Colors.white,
            child: Icon(Icons.person),
          ),
          title: Text('Senai', style: TextStyle(color: Colors.white)),
          //subtitle: Text('last seen today at 13:25',
            //  style: TextStyle(color: Colors.white70, fontSize: 12)),
        ),
        backgroundColor: Colors.teal,
      ),
```

Flutter Widgets - Tela ChatBot

```
body: Column(
        children: [
          Expanded(
            child: ListView.builder(
              padding: const EdgeInsets.all(10),
              itemCount: _messages.length,
              itemBuilder: (context, index) {
                final msg = _messages[index];
                return MessageBubble(
                  text: msg['text'],
                  isMe: msg['isMe'],
                  time: msg['time'],
                );
              },
            ),
          ),
```

Flutter Widgets - Tela ChatBot

```
Padding(
            padding: const EdgeInsets.symmetric(horizontal: 8,
vertical: 4),
            child: Row(
              children: [
                Expanded(
                  child: TextField(
                    controller: _controller,
                    decoration: const InputDecoration(
                      hintText: "Digite sua mensagem ",
                      border: InputBorder.none,
                    ),
                  ),
                ),
```

Flutter Widgets - Tela ChatBot

```
 IconButton(
                    icon: const Icon(Icons.send, color: Colors.teal),
                    onPressed: _sendMessage,
                  ),
                IconButton(onPressed: _limparMessages, icon:
Icon(Icons.cleaning_services,color: Colors.teal,))
                ],
              ),
            ),
          ],
        ),
      );
  }
}
```

Flutter Widgets - Tela ChatBot

```
class MessageBubble extends StatelessWidget {
  final String text;
  final bool isMe;
  final String time;

  const MessageBubble({
    super.key,
    required this.text,
    required this.isMe,
    required this.time,
  });
```

Flutter Widgets - Tela ChatBot

```
@override
 Widget build(BuildContext context) {
   return Align(
     alignment: isMe ? Alignment.centerRight : Alignment.centerLeft,
     child: Column(
       crossAxisAlignment:
           isMe ? CrossAxisAlignment.end : CrossAxisAlignment.start,
       children: [
         Container(
           margin: const EdgeInsets.symmetric(vertical: 5),
           padding: const EdgeInsets.all(12),
           decoration: BoxDecoration(
             color: isMe ? Colors.green[100] : Colors.grey[300],
             borderRadius: BorderRadius.circular(12),
           ),
           child: Text(text),
         ),
```

Flutter Widgets - Tela ChatBot

```
Text(
            time,
            style: const TextStyle(fontSize: 10, color: Colors.grey),
          ),
        ],
      ),
    );
  }
}
```

Criando projeto no VSCODE

Para criar um projeto de aplicativo mobile utilizando o framework Flutter com a IDE VSCode é necessário seguir os passos listados a seguir:

1º Abrir o VSCode

2º Instalar os plugins.

Criando projeto no VSCODE

File Edit Selection View Go Run Terminal Help

Untitled (Workspace)

EXTENSIONS: MARKET...

flutter

Flutter
Flutter support and debugg...
Dart Code

Flutter Widg... 1.6M ★ 4.5
A set of helpful widget snip...
Alexis Villegas Torres Install

Awesome Flut... 727K ★ 5
Awesome Flutter Snippets is...
Neevash Ramdial (N... Install

Flutter Tree 464K ★ 5
Extension for Flutter to buil...
Marcelo Velasquez Install

[FF] Flutter Files 527K ★ 5
Quickly scaffold flutter bloc ...
Igor Kravchenko Install

Flutter Color 400K ★ 4.5
This plugin help you to visu...
Nilesh Chavan Install

Flutter Intl 366K ★ 5
Flutter localization binding f...
Localizely Install

Flutter Helpers 300K ★ 5
Helper utilities for flutter pr...
Akshar Patel Install

flutter-stylizer 202K ★ 4.5
Flutter Stylizer organizes yo...
gmlewis-vscode Install

Flutter & Dart... 177K ★ 5
Official package of Academi...
Rodrigo Rahman Install

Flutter Snippets 133K ★ 5
Supercharge your Flutter w...
Maruf Hassan Install

vscode-flutter... 132K ★ 5
VS Code extension to create...

Extension: Flutter

ESP-IDF: Search Error Hint

Flutter v3.100.0

Dart Code dartcode.org

Flutter support and debugger for Visual Studio Code.

Disable | Uninstall | Switch to Pre-Release Version

This extension is enabled globally.

DETAILS FEATURE CONTRIBUTIONS CHANGELOG DEPENDENCIES RUNTIME STATUS

Introduction

This VS Code extension adds support for effectively editing, refactoring, running, and reloading Flutter mobile apps. It depends on (and will automatically install) the Dart extension for support for the Dart programming language.

Note: Projects should be run using `F5` or the `Debug` menu for full debugging functionality. Running from the built-in terminal will not provide all features.

Installation

Install from the Visual Studio Code Marketplace or by searching within VS Code. The Dart extension will be installed automatically, if not already installed.

Documentation

Please see the Flutter documentation for using VS Code.

Reporting Issues

Issues for both Dart and Flutter extensions should be reported in the Dart-Code issue tracker.

Categories

Programming Languages | Snippets | Linters | Formatters | Debuggers

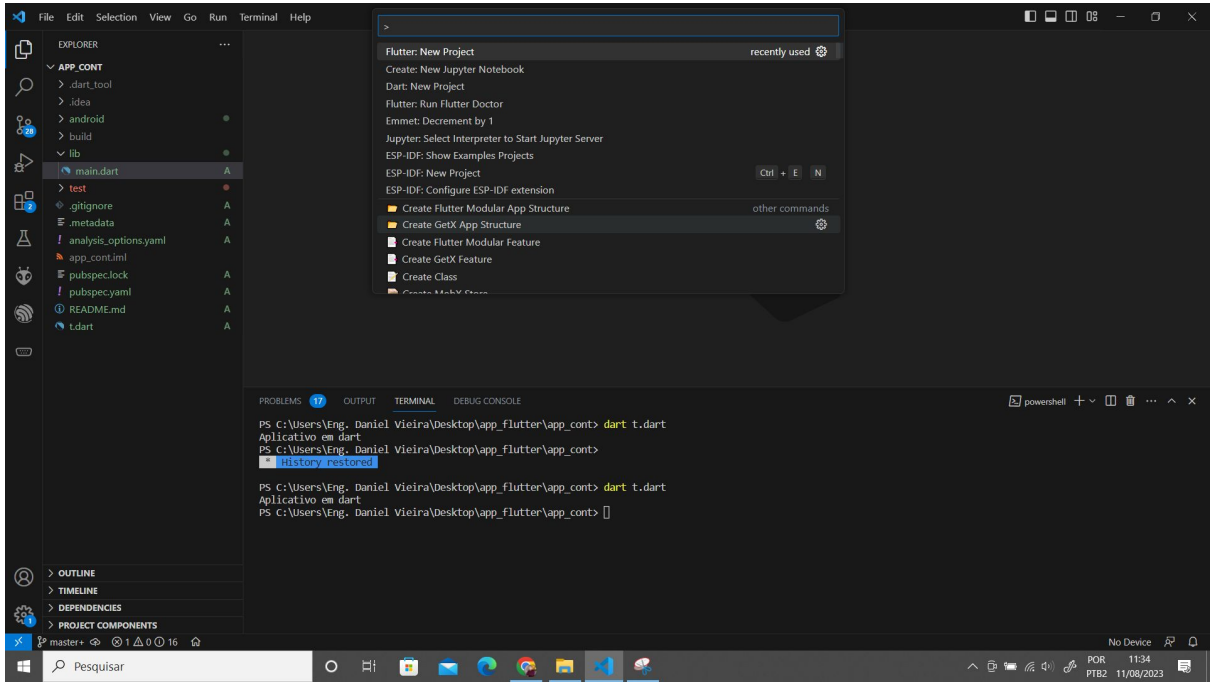
More Info

Last updated	2024-11-03, 20:29:04
Identifier	`dart-code.flutter`

main 0 0 0 Discovering Python Interpreters

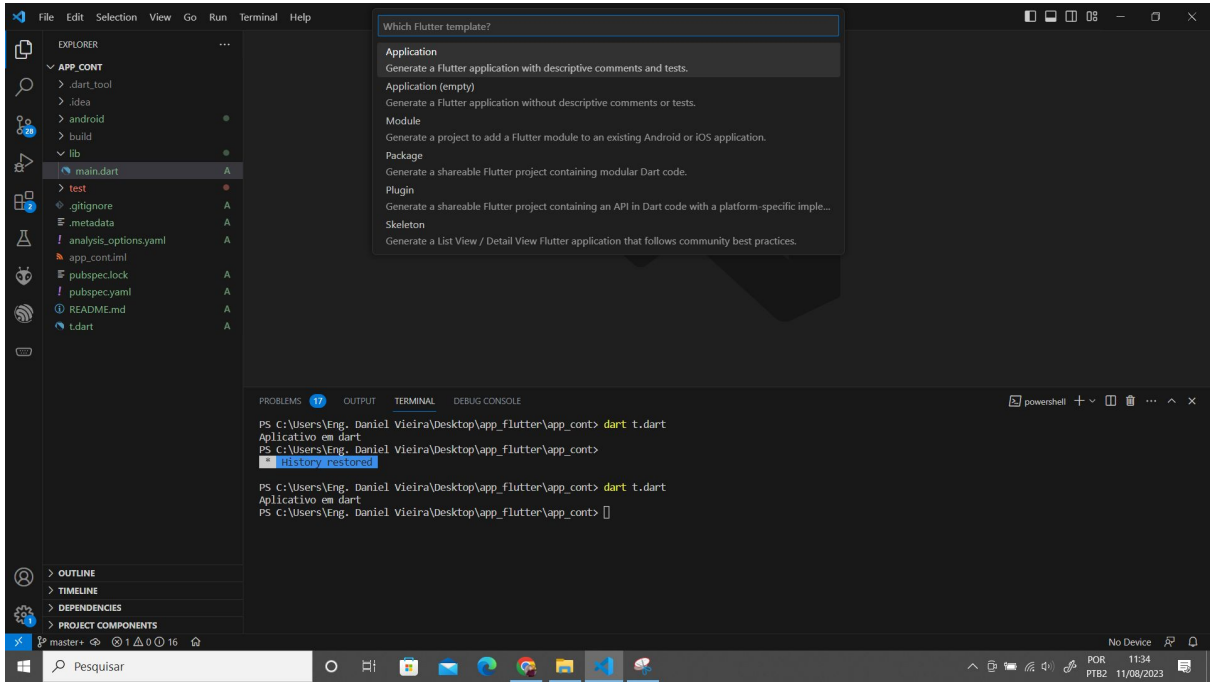
Criando projeto no VSCODE

3º Apertar a tecla F1, após apertar em F1 clicar em Flutter New Project conforme



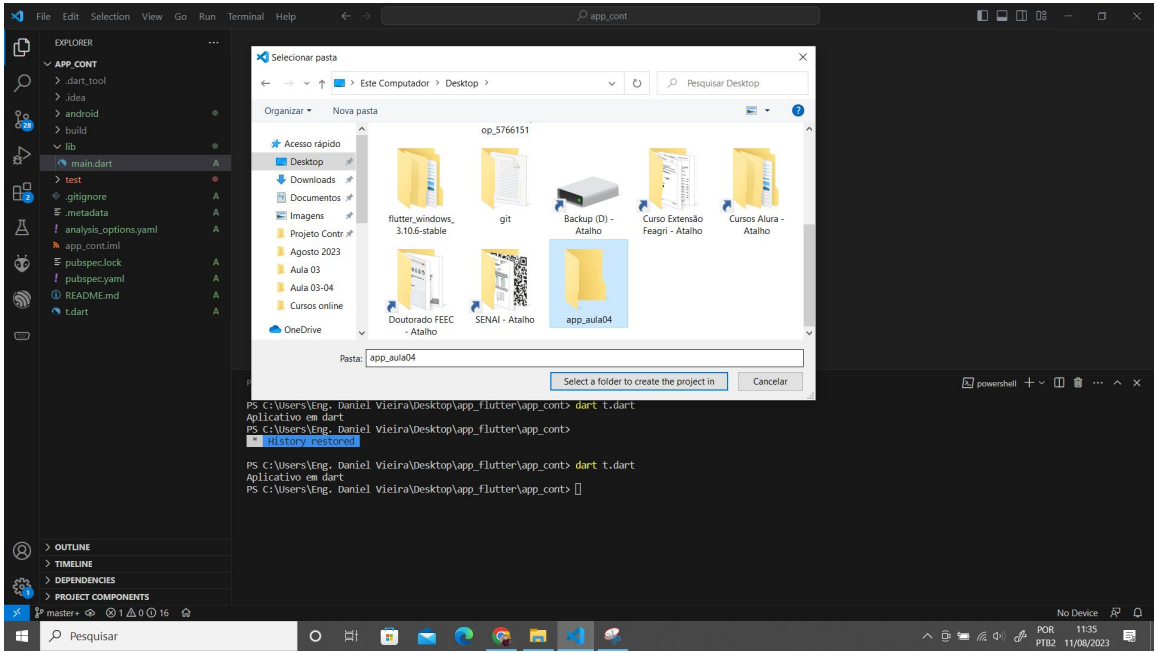
Criando projeto no VSCODE

4º Selecionar a opção Application conforme a figura abaixo



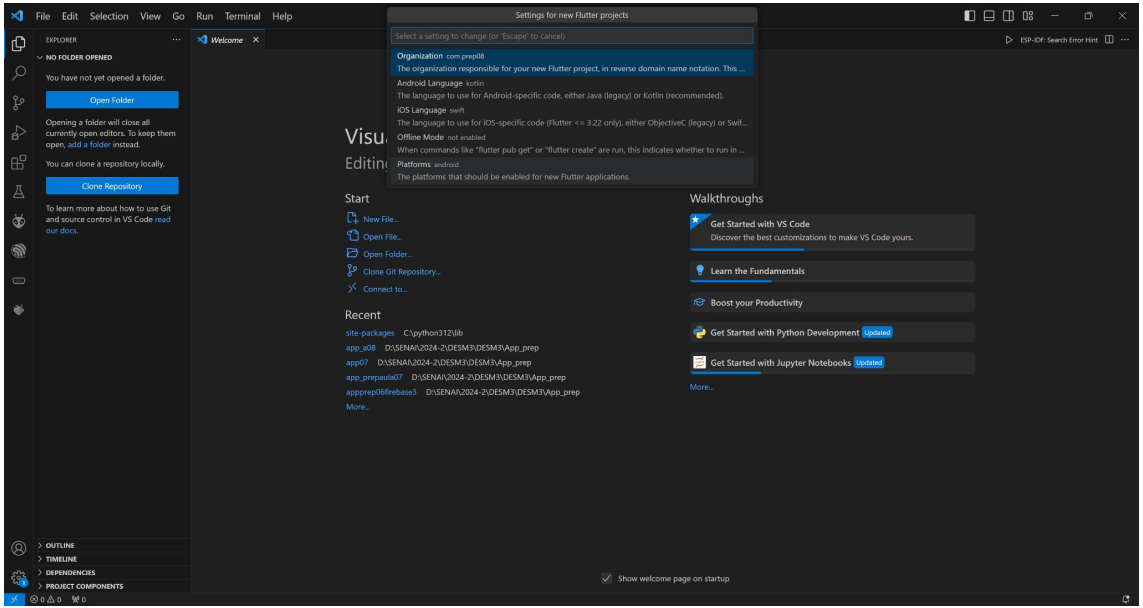
Criando projeto no VSCODE

5º Selecionar uma pasta para salvar o projeto conforme a Figura. A pasta não deve começar com letras maiúsculas, números, não pode ter espaço no nome da pasta



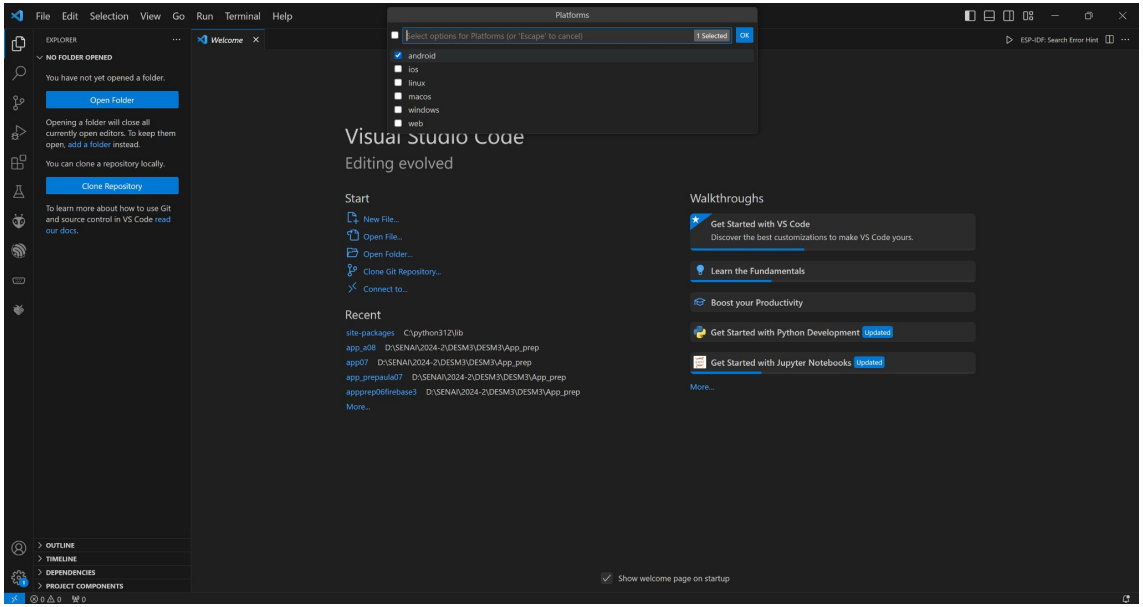
Criando projeto no VSCODE

6º Clicar na engrenagem para configurar as plataformas mobile que o projeto será executado conforme a Figura



Criando projeto no VSCODE

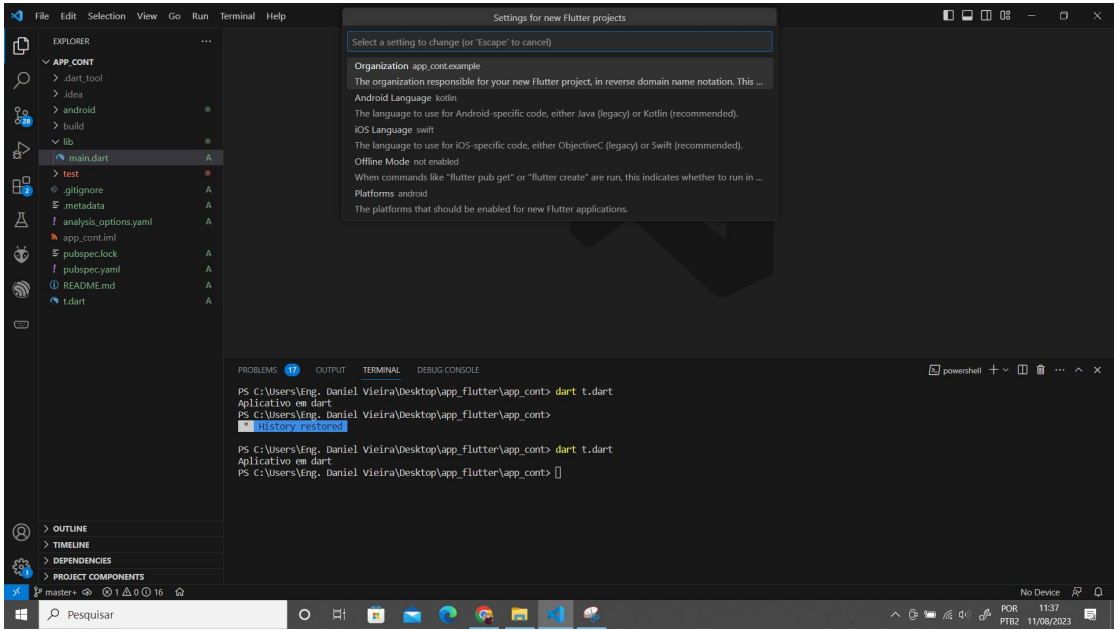
7º Selecionar a opção Android, realizada essa opção apertar enter conforme a Figura



Criando projeto no VSCODE

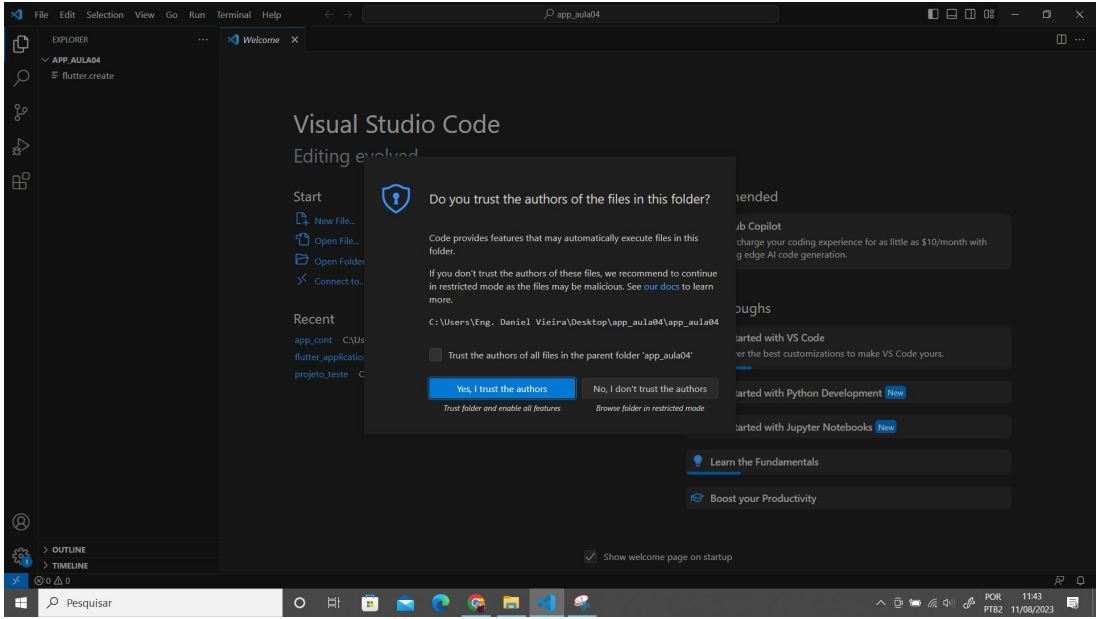
8º Passo Clicar em Organization conforme a Figura 48

Organization é o parâmetro que indica o domínio da empresa que desenvolveu o app. É um parâmetro importante para publicação do APP em lojas virtuais.



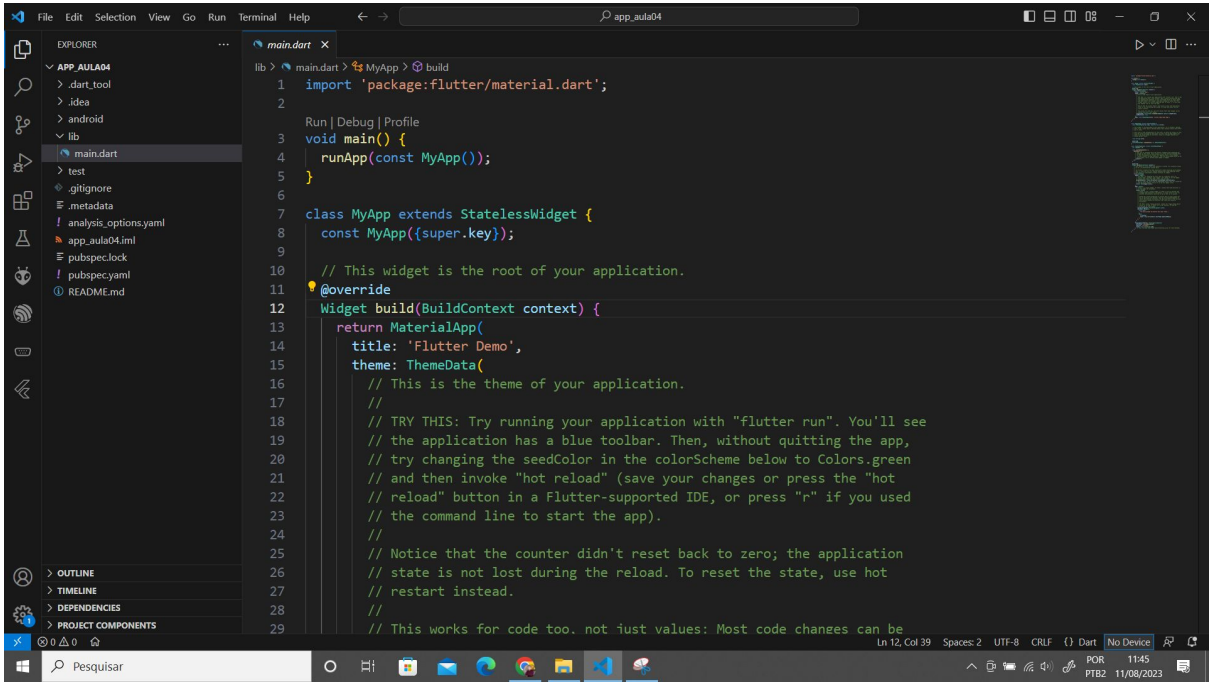
Criando projeto no VSCODE

Após realizar as configurações, apertar Esc e enter
O projeto Flutter será criado Marcar a opção Yes, I Trust the authors conforme a Figura.



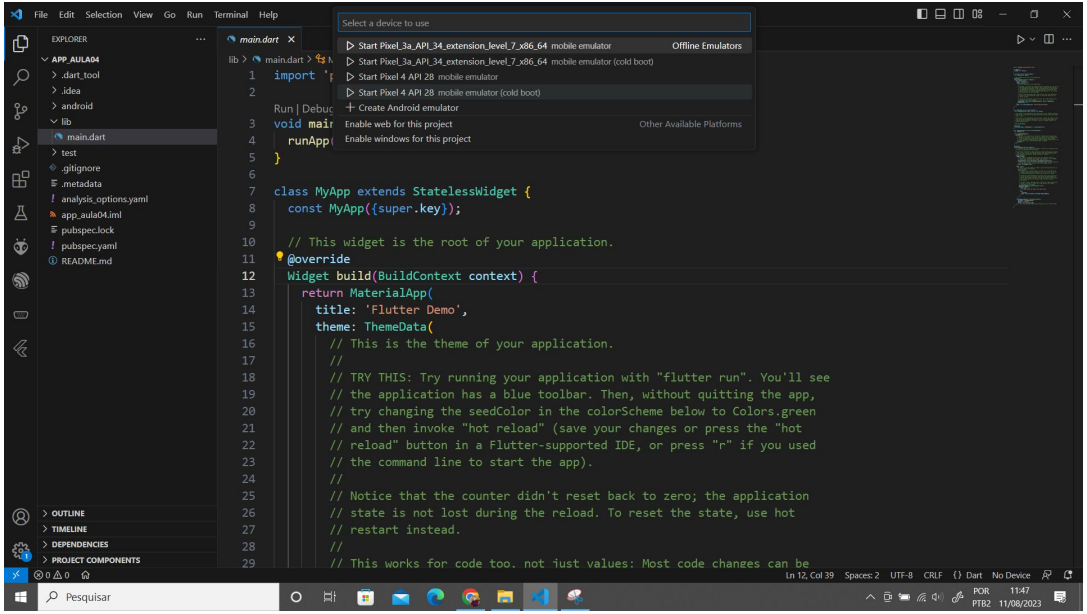
Criando projeto no VSCODE

Para executar o aplicativo no emulador o primeiro passo é clicar em iremos escolher o device para emular nosso telefone, clicar em No Device e selecionar o emulador desejado conforme a Figura.



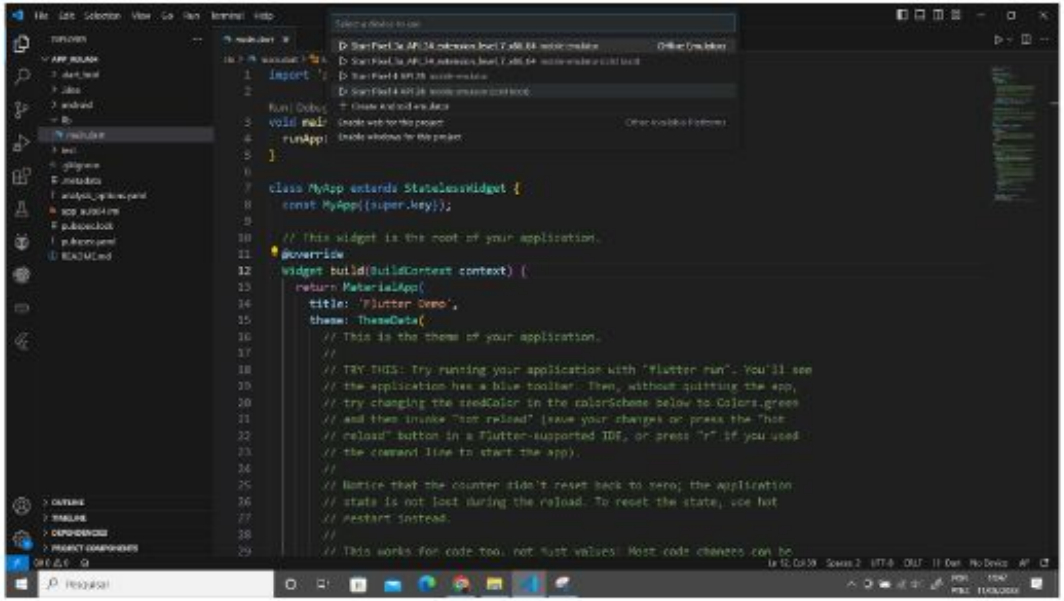
Criando projeto no VSCODE

Após clicar em No Device no editor VSCode irá aparecer para escolher o emulador para executar o aplicativo conforme a Figura, caso não apareça nenhum emulador, então é necessário criá-lo.



Criando projeto no VSCODE

Na Figura, após selecionar o device ele irá aparecer na tela

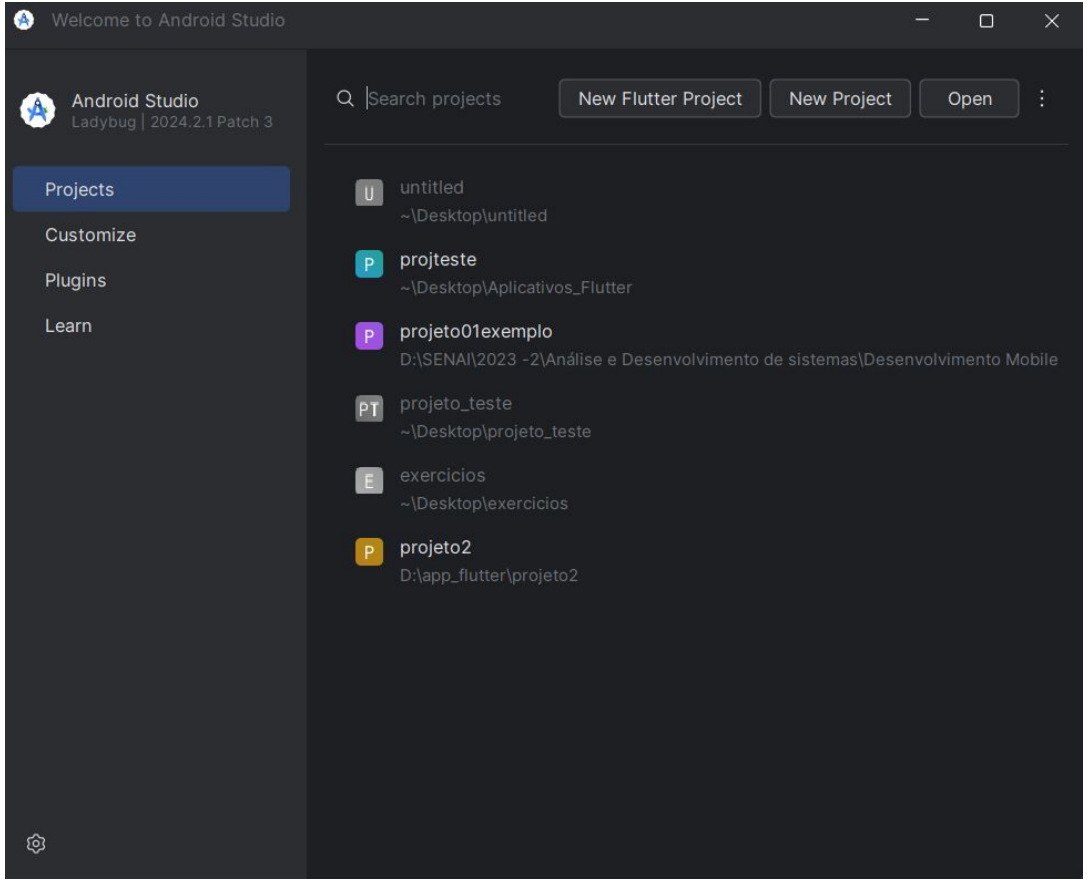


Criando emulador Android

Uma forma muito eficiente e rápida de testar os aplicativos desenvolvidos é utilizando o emulador do Android Studio, para isso é necessário criá-lo no Android Studio conforme os passos a seguir :

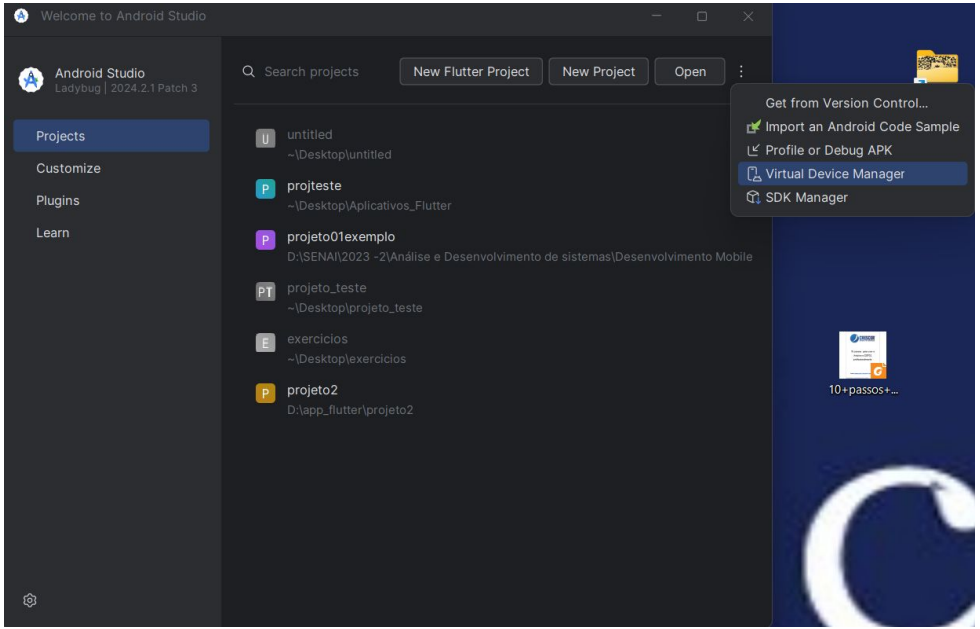
1º Abrir o Android Studio conforme a Figura 54 e clicar nos três pontos ao lado do botão Open

Criando emulador Android



Criando emulador Android

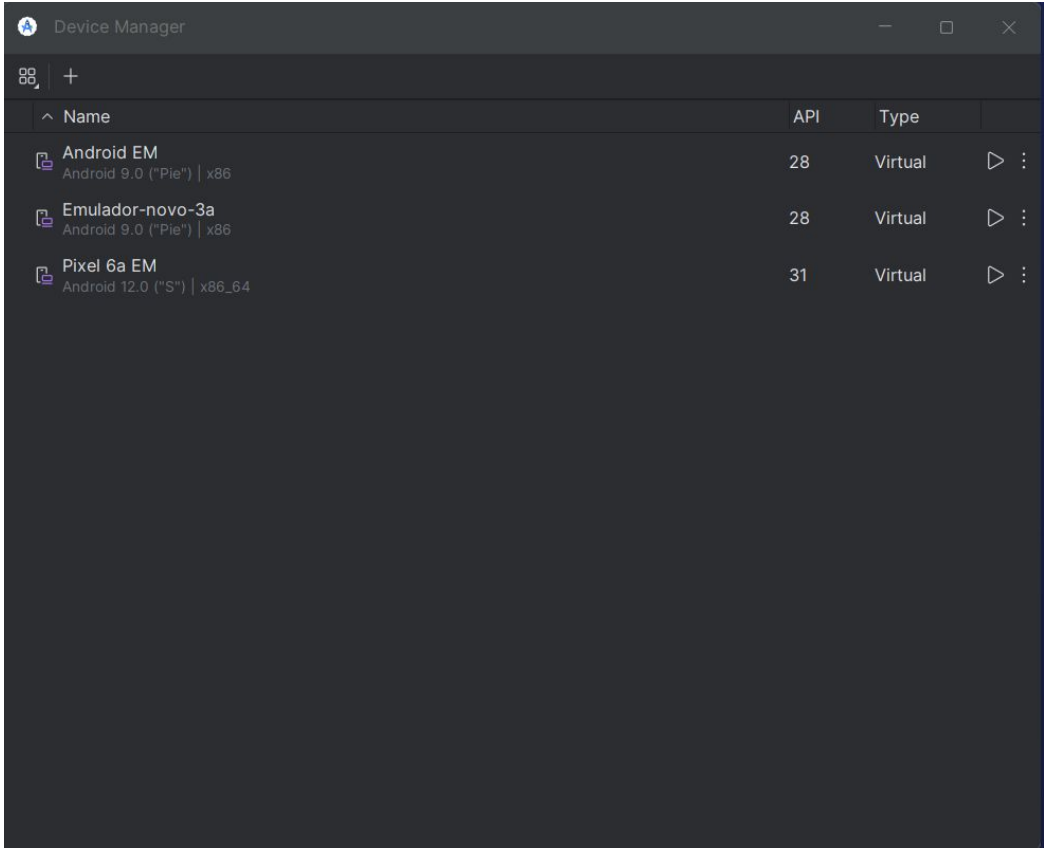
Na Figura após clicar nos três pontos ao lado do botão open, clicamos em Virtual Device Manager



Criando emulador Android

Após clicar em Virtual Device Manager, a tela com os emuladores criados irá aparecer conforme a Figura.

Criando emulador Android

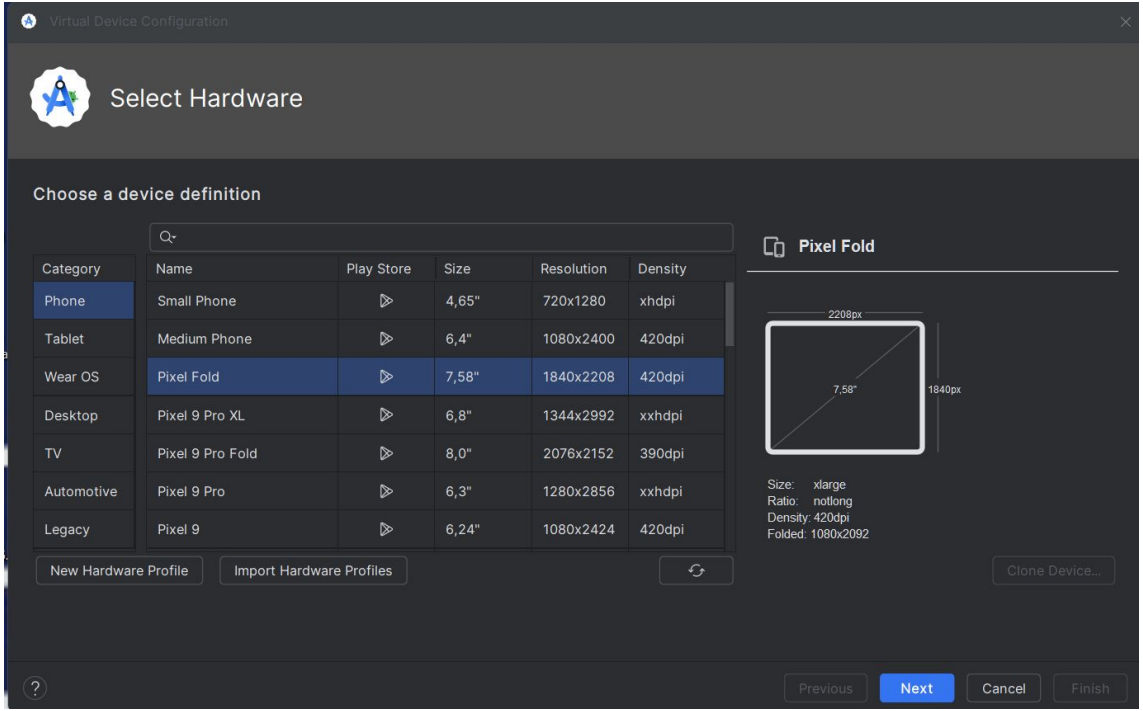


Criando emulador Android

Para criar um novo emulador é necessário clicar no ícone +

Após clicar no ícone + para criar um novo emulador irá aparecer a lista dos dispositivos emuladores disponíveis para serem escolhidos conforme a Figura

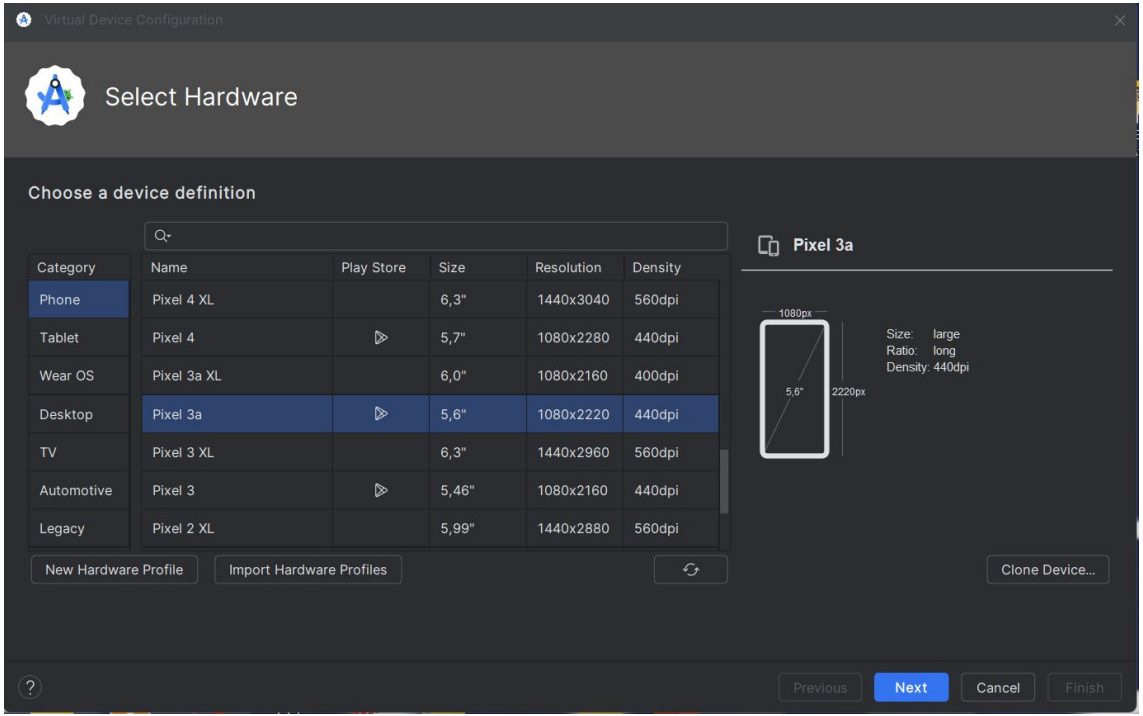
Criando emulador Android



Criando emulador Android

Iremos escolher o emulador Pixel 3a conforme a Figura, ou qualquer outro emulador que esteja com o ícone da Play Store ativado, para caso nós desejemos hospedar o aplicativo na Play Store.

Criando emulador Android



Criando emulador Android

Com o emulador Pixel 3a escolhido, o próximo passo será a escolha da versão do sistema operacional conforme a Figura.

A versão escolhida será a versão Pie por ser mais leve para ser executada em nosso dispositivo.

Criando emulador Android

Virtual Device Configuration

System Image

Select a system image

Recommended | x86 Images | Other Images

Release Name	API	ABI	xABI	Target
Sv2	32	x86_64		Android 12L (Google Play)
S	31	x86_64		Android 12.0 (Google Play
R	30	x86		Android 11.0 (Google Play)
Q	29	x86		Android 10.0 (Google Play
Pie	28	x86		Android 9.0 (Google Play)
Oreo	27	x86		Android 8.1 (Google Play)
Oreo	26	x86		Android 8.0 (Google Play)
Nougat	25	x86		Android 7.1.1 (Google Play
Nougat	24	x86		Android 7.0 (Google Play)

Pie

API Level
28

Type
Google Play

Android
9.0
Google Inc.

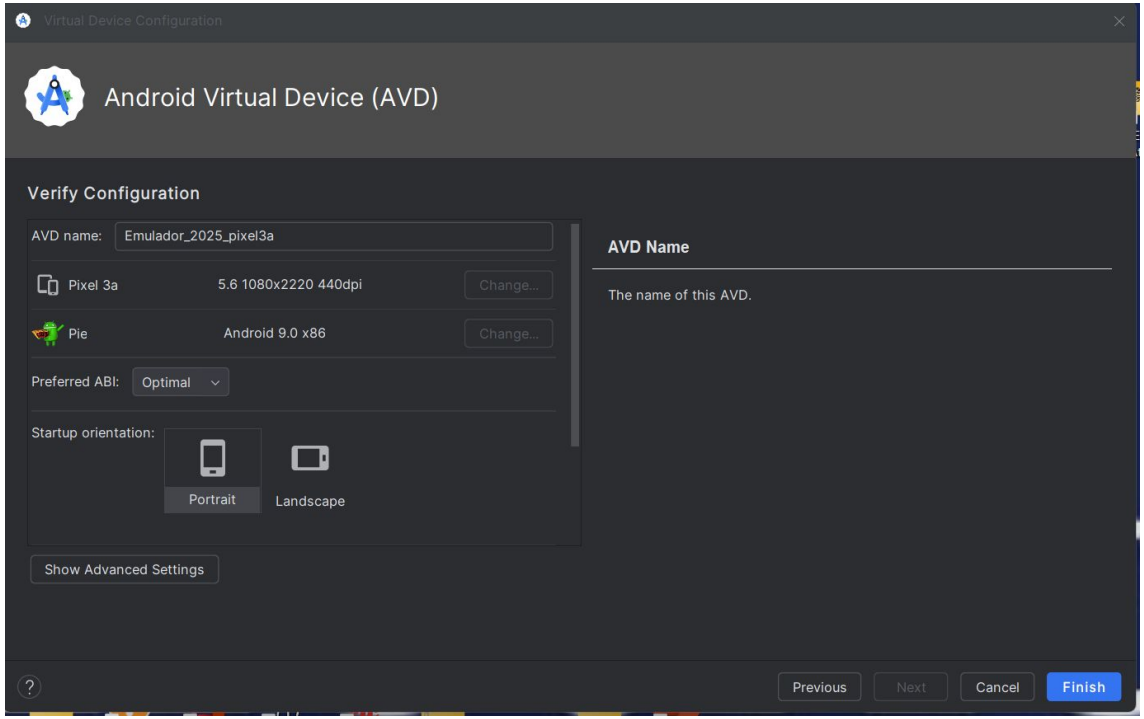
System Image
x86

We recommend these Google Play images because this device is compatible with Google Play

Previous | Next | Cancel | Finish

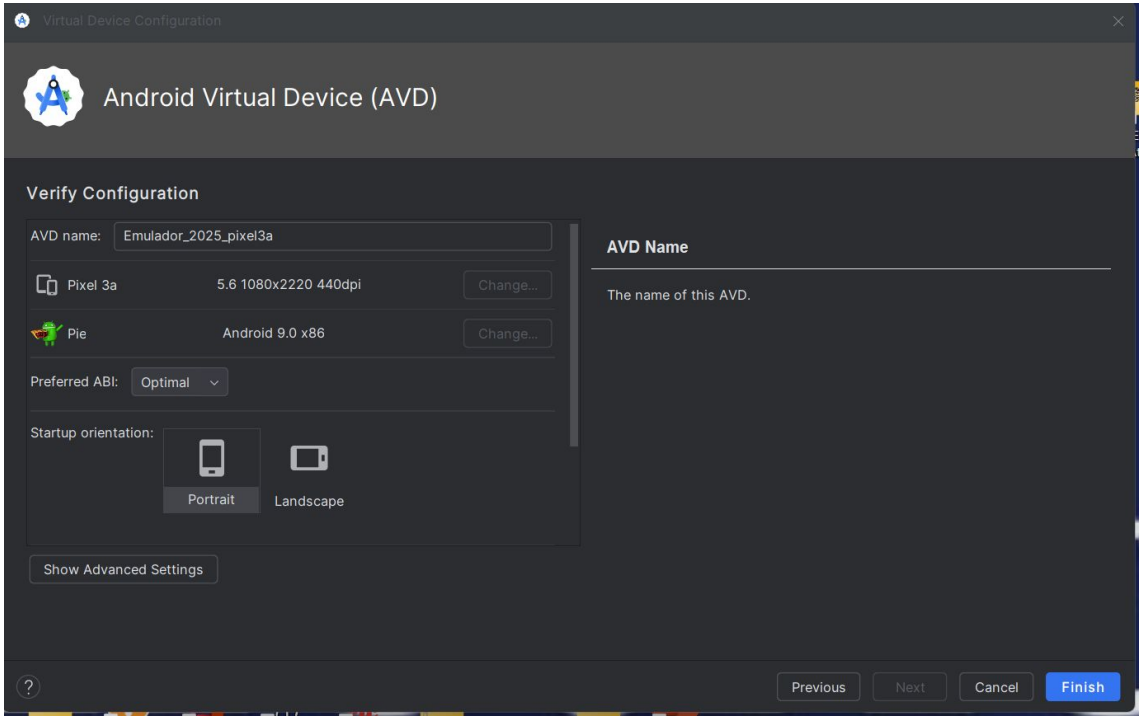
Criando emulador Android

Clicar em Next e nomear o emulador conforme a Figura



Criando emulador Android

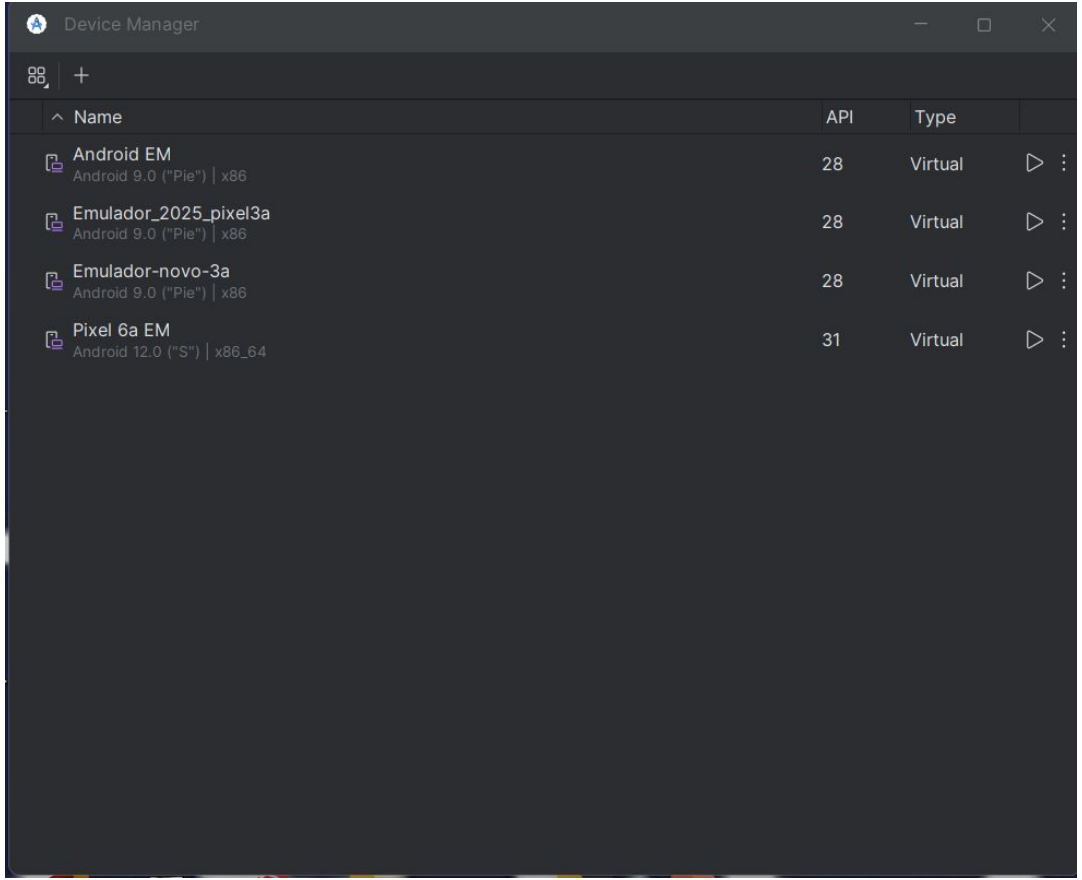
Após atribuir o nome Emulador_2025_pixel3a ao emulador clicar em Finish e o emulador será criado.



Criando emulador Android

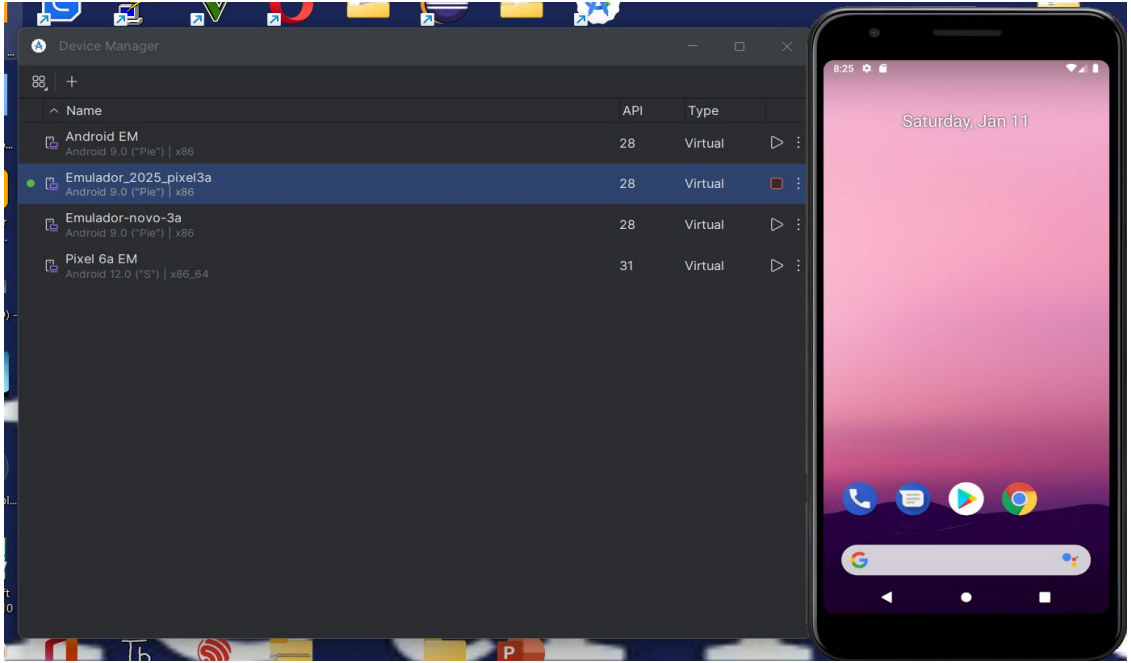
Na Figura é possível visualizar o emulador criado e é só clicar no play para o emulador ser inicializado abrindo o celular na Tela conforme a Figura

Criando emulador Android



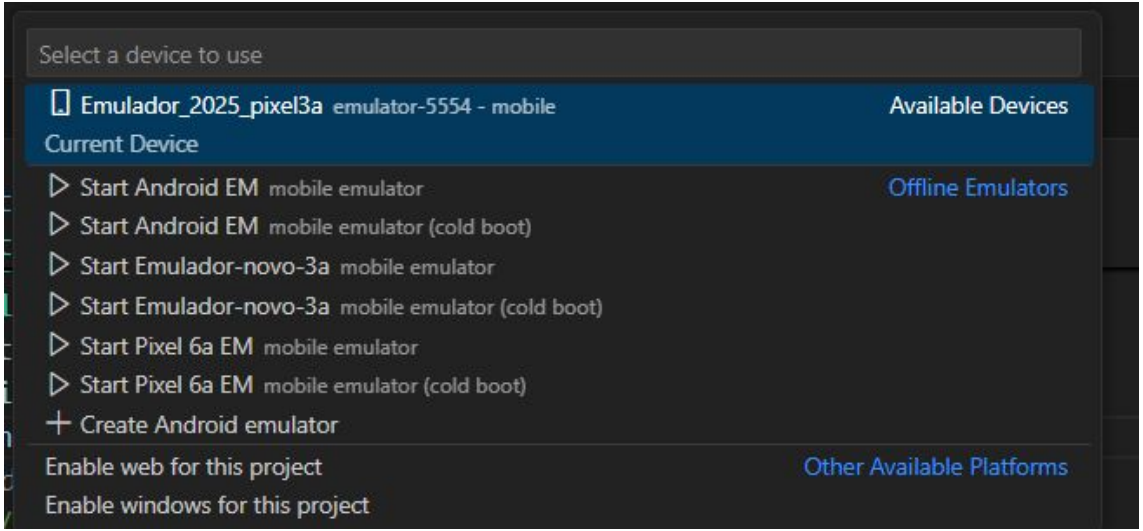
Criando emulador Android

Na Figura é possível ver o emulador sendo executado e agora é possível utilizá-lo para executar os aplicativos.



Criando emulador Android

Pronto, agora temos o emulador criado para executar os nossos aplicativos.



Exercício

1) Desenvolva um aplicativo com um Navigator Bar que contenha 4 opções representando diferentes países (por exemplo: Brasil, Itália, Japão, México).

   Em cada tela, exiba informações sobre uma comida típica do país (incluindo o nome da comida e uma imagem).

   Exemplo de saída:

   Brasil: Feijoada (com imagem).

   Itália: Pizza (com imagem).

   Japão: Sushi (com imagem).

   México: Tacos (com imagem).

Exercício

2) Crie um aplicativo para coletar informações de um usuário.

Perguntas obrigatórias:

Nome

Idade

Profissão

Sexo: Use Radio Buttons para as opções Masculino e Feminino.

Estado Civil: Use Radio Buttons para as opções Solteiro, Casado, Divorciado e Viúvo.

Após o preenchimento, exiba os dados coletados em uma tela separada (usando o Navigator).

Exemplo de fluxo:

O usuário seleciona as opções desejadas e pressiona um botão "Concluir".

Na próxima tela, os dados escolhidos são exibidos.

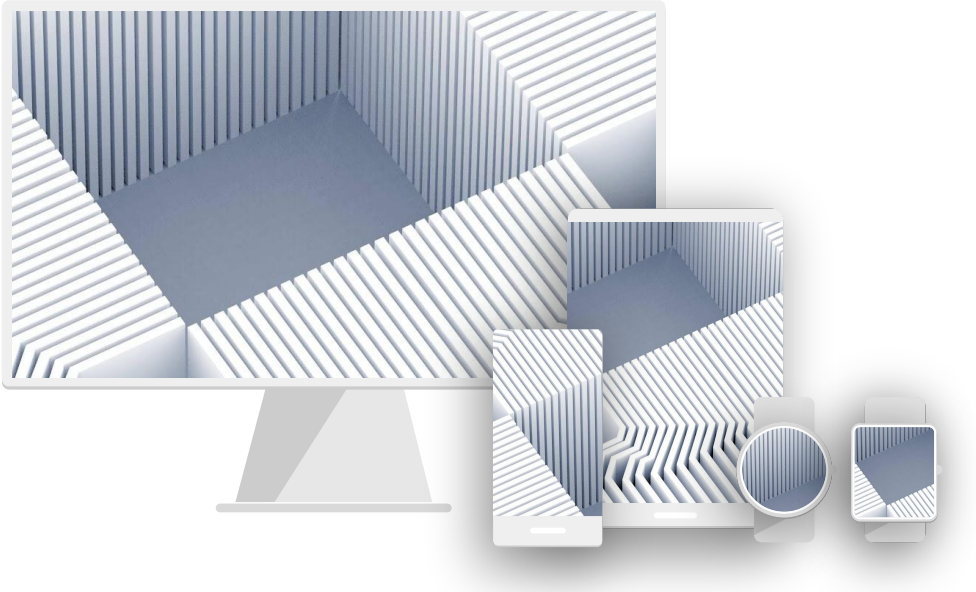
Exercício

Entrega dos 2 exercícios via TEAMS através do GITHUB, não esquecer de me adicionar como colaborador caso o repositório esteja privado

Obrigado!

Prof. Me Daniel Vieira

Email: danielvieira2006@gmail.com

Linkedin: Daniel Vieira

Instagram: Prof daniel.vieira95